

1. Termin Wahrscheinlichkeitsrechnung

Studiengang: Bachelor Elektronik und Informationstechnologie Dual

Datum: 22.12.2025

Zeit: 09:50 bis 11:25

Hilfsmittel: 2 Seiten A4, selbstbeschrieben

nicht erlaubt: Mobiltelefon, Smart Watch, Taschenrechner, Unterlagen aus der Vorlesung

Hinweise: Bitte stelle deine Rechenwege nachvollziehbar. Verwende bei den Grafiken sinnvolle Beschriftungen und basiere deine Interpretationen anhand der ermittelten statistischen Größen oder Darstellungen. Du kannst neben Einträgen in dieses Notebook, auch Antworten und Rechenwege auf einem Blatt Papier erstellen.

Schreibe bitte deinen Namen auf das Blatt Papier.

Viel Erfolg!

Bitte führe die nachstehende Zelle aus, um die notwendigen Module zu laden.

```
In [1]: import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sn
import statsmodels.api as sm
from scipy.stats import norm
from scipy.stats import binom
from scipy.stats import hypergeom
from scipy.stats import t
from scipy.stats import chi2
from scipy.stats import contingency

import pingouin as pg

#from sklearn import datasets
from sklearn.cluster import KMeans # kMeans
from sklearn.preprocessing import StandardScaler # scaling data
from sklearn.metrics import silhouette_score, silhouette_samples, r2_score #
from sklearn.model_selection import KFold, train_test_split

from yellowbrick.cluster import SilhouetteVisualizer # validation
plt.rcParams['font.family'] = 'monospace' # backup, falls fonts nicht gefunc
```

```
import warnings
warnings.filterwarnings('ignore')
```

1) Beschreibende Statistik - Univariater Fall [8 Punkte]

Lade den *Christmas Sales and Trends.csv* Datensatz als DataFrame ein.

- Welche Merkmale des Datensatzes sind metrische Merkmale? [2 Punkte]
- Erstelle ein kumuliertes Häufigkeitsdiagramm für das Merkmal *TotalPrice*. Welchen Mindestwert haben die oberen 10% der Ausprägungen für dieses Merkmal? [2 Punkte]
- Gib die Definitionen der einzelnen Elemente eines Boxplots an. Stelle anschließend die Boxplots für das Merkmal *DiscountAmount* für die verschiedenen Events im Merkmal *Event* dar. Welche Interpretationen lassen sich aus der Grafik ableiten? [4 Punkte]

```
In [33]: ChristmasSales = pd.read_csv("Christmas Sales and Trends.csv")
#display(ChristmasSales.head(3))
#display(ChristmasSales.tail(3))
#display(ChristmasSales.info)
#b)

ChristmasSalesTotalPrice = ChristmasSales['TotalPrice']
display(ChristmasSalesTotalPrice.head(5))

#fig, ax = plt.subplots(figsize=(10,9))
#sn.histplot(data=ChristmasSalesTotalPrice, x='runtime',stat='percent', ax,
#ax.set_title('Kumulative')
#ax.grid()
#plt.show

#c)
#ChrstSales_DiscountAmount = ChristmasSales['DiscountAmount']
#fix, ax
```

Cell In[33], line 11

```
sn.histplot(data=ChristmasSalesTotalPrice, x='runtime',stat='percent', a
x, cumulative=True, element='step', fill = False, label = 'histplot')
```

^

SyntaxError: positional argument follows keyword argument

a)

c) Bei den Boxplots können die Quartile Abgelesen werden und zusätzlich noch Ausreißer siehe Zettel Im boxplot ist der Median (also wo 50% der Daten liegt)

erkennbar und die jeweiligen Quartile also wo wie viel Prozent der Daten liegt. Wenn man es mit Quartilen macht sind es 25% je Quartil.

2) Beschreibende Statistik - Multivariater Fall [6 Punkte]

Es wird wiederum der Datensatz *Christmas Sales and Trends.csv* verwendet.

a) Stelle die Kreuztabelle für die Merkmale *GiftWrap* und *Category*, inklusive der entsprechenden Randhäufigkeiten, dar. [1 Punkt]

b) Bestimme folgende bedingte Häufigkeiten: [3 Punkte]

- Das ein Artikel aus der Kategorie **Food** eingepackt (GiftWrap = True) wird.
- Das ein unverpackter Artikel aus der Kategorie **Decorations** stammt.
- Das ein eingepackter Artikel ***nicht*** zur Kategorie **Toys** gehört.

c) Bestimme den Korrigierten Kontingenzkoeffizient für die Merkmale *Weather* und *Category*. Welche Aussage lässt sich aus dem Ergebnis ableiten? [2 Punkte]

```
In [34]: # Code Snippet für c)
# ktab ist die zugehörige Kreuztabelle / Kontingenztabelle
pearson = contingency.association(ktab, method='pearson')
nExpr = min(ktab.shape)
print('Pearson korrigiert: {0:.4f} (max. 1)'.format(pearson*np.sqrt(nExpr/(r
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[34], line 3
      1 # Code Snippet für c)
      2 # ktab ist die zugehörige Kreuztabelle / Kontingenztabelle
----> 3 pearson = contingency.association(ktab, method='pearson')
      4 nExpr = min(ktab.shape)
      5 print('Pearson korrigiert: {0:.4f} (max. 1)'.format(pearson*np.sqrt(
(nExpr/(nExpr-1))))

NameError: name 'ktab' is not defined
```

```
In [42]: #a)
ChrSal = ChristmasSales
kreuztab = pd.crosstab(ChrSal['GiftWrap'],ChrSal['Category'], margins=True)
print(kreuztab)

#b)
h1 = 976
total1 = 1991
```

```
print(f"P(GiftWrapTrue|Food)= {(h1/total1)*100:.2f}%")

h2 = 970
total2 = 1975
print(f"P(Decorations|GiftWrapFalse)= {(h2/total2)*100:.2f}%")

print(f"Häufigkeit dass eingepackter Artikel nicht zur Kategorie Toys gehört
```

Category	Clothing	Decorations	Electronics	Food	Toys	All
GiftWrap						
False	986	970	1021	1015	983	4975
True	964	1025	1032	976	1028	5025
All	1950	1995	2053	1991	2011	10000

P(GiftWrapTrue|Food)= 49.02%

P(Decorations|GiftWrapFalse)= 49.11%

Häufigkeit dass eingepackter Artikel nicht zur Kategorie Toys gehört = 19.56%

4) Erwartungswerte von Zufallsvariablen [4 Punkte]

Auf dem Martinimarkt entdeckst du folgendes Glücksspiel. Für einen Einsatz von 3 Euro kann der Spieler einmal mit zwei Würfeln würfeln. Bei der Augensumme 11 und 12 erhält der Spieler jeweils 15 Euro und bei der Augensumme 2 erhält der Spieler 10 Euro. In allen anderen Fällen ist der Einsatz verloren. Bestimme den zu erwartenden Gewinn/Verlust. Wie ist dieser zu interpretieren?

```
In [8]: Ergebnis = []
for w1 in range(1,7):
    for w2 in range(1,7):
        Ergebnis.append(w1+w2)
Augensummen = pd.Series(Ergebnis).value_counts().sort_index()

x = Augensummen.index.values
absH = Augensummen.values
relH = absH/36

dataWürfel = pd.DataFrame(data={
    "Summe Augenzahl":x,
    "Abs. Häufigkeit":absH,
    "Rel. Häufigkeit":relH
})

print(dataWürfel)
#siehe Handschriftlicher Zettel für weitere Rechnung bzw Angabe über Nettogewinn
Erwartungswert = (7*0.027-3*0.055-3*0.0833-3*0.111-3*0.1388-3*0.1666-3*0.1388)
print(f"Der Erwartungswert ergibt: {Erwartungswert:.4f}€")
```

	Summe	Augenzahl	Abs. Häufigkeit	Rel. Häufigkeit
0		2	1	0.027778
1		3	2	0.055556
2		4	3	0.083333
3		5	4	0.111111
4		6	5	0.138889
5		7	6	0.166667
6		8	5	0.138889
7		9	4	0.111111
8		10	3	0.083333
9		11	2	0.055556
10		12	1	0.027778

Der Erwartungswert ergibt: -1.4904€

Da der Erwartungswert -1.4904€ ergeben hat, macht der Spieler je öfter er spielt einen Verlust um den Betrag.

5) Wichtige Sätze der Stochastik [2 Punkt]

Erkläre in eigenen Worten, was der zentrale Grenzwertsatz aussagt.

6) Schließende Statistik [8 Punkte]

Lade den Datensatz *Rotorblätter*. Es wurden die Länge [m] und das Gewicht [t] der produzierten Rotorblätter für Windräder aufgezeichnet.

a) Visualisiere die Zufallsvariable $X = \text{"length"}$ in einem Boxplot. Ordne jeder Gruppe eine andere Farbe zu. Was schließt du daraus? [2 Punkt]

b) Bestimme jeweils die Konfidenzintervalle für den Erwartungswert von der Variable *length* (pro Gruppe) zum Niveau $\alpha = 0.05$. Was bedeuten diese? Vergleiche die Konfidenzintervalle, was schließt du daraus? [4 Punkte]

c) Für den EU-Markt gilt ein Sollwert der Rotorblattlänge von 115 m. Prüfe zum Signifikanzniveau $\alpha = 0.05$, ob die Produktion neu kalibriert werden muss. [2 Punkt]

```
In [69]: Rotor = pd.read_csv('Rotorblätter.csv')
display(Rotor.head(3))

fig, axs = plt.subplots(figsize=(8,5))
sn.boxplot(data=Rotor, y=feature)
plt.show
```

	Unnamed: 0	length	weight	group
0	0	113.592145	54.956861	EU
1	1	115.806633	57.101781	EU
2	2	115.214354	62.480009	EU

7) Regression [6 Punkte]

Lade den Datensatz *College.csv* ein.

a) Bestimme die 3 Merkmale mit der stärksten Korrelation für das Merkmal *Apps* (Applications received). [1 Punkt]

b) Erstelle ein Regressionsmodell für das Merkmal *Apps* basierend auf allen anderen Merkmalen des Datensatzes. Unterteile dazu den Datensatz zuerst in einen Trainings- und einen Testdatensatz. Eine Skalierung der Daten ist jedoch nicht notwendig. Welchen Aussagen über Qualität der Linearen Regression erhältst du und welche Interpretationen lassen sich daraus ableiten? [4 Punkte]

c) Auf dem Testdatensatz ergibt sich $R_{adj}^2 = 0.91$. Was lässt sich aus diesem Wert ableiten? [1 Punkte]

```
In [ ]: # Code Snippet für Regression
# var_u - unabhängige Variablen
# var_a - abhängige Variable
x_train, x_test, y_train, y_test = train_test_split(var_u, var_a, train_size=
var = sm.add_constant(x_train,prepend=True)
model = sm.OLS(y_train, var)
res = model.fit()
```

```
In [55]: College = pd.read_csv("College.csv")
display(College.head(3))

x = College[['Top10perc', 'Top25perc', 'F.Undergrad', 'P.Undergrad', 'Outsta
y = College['Apps']

x_const = sm.add_constant(x)
model_multi = sm.OLS(y,x_const)
res_multi = model_multi.fit()
print(res_multi.summary())

#regression der anderen Merkmale

col = 'Apps'
target = 'Top10perc'
x_train, x_test, y_train, y_test = train_test_split(College[col], College[ta
var = sm.add_constant(x_train,prepend=True)
```

```

model = sm.OLS(y_train, var)
res = model.fit()
pred = res.predict(sm.add_constant(x_test, prepend=True))
fig, axs = plt.subplots(figsize=(8,5))
sn.scatterplot(x=x_train, y=y_train, ax=axs, alpha=1, label='training')
sn.lineplot(x=x_train, y = y_train, ax = axs, label='model', linewidth = 4)
sn.scatterplot(x=x_test, y=y_test, ax=axs, label='test')
plt.show

```

	Unnamed: 0	Private	Apps	Accept	Enroll	Top10perc	Top25perc	F.Undergra
0	Abilene Christian University	Yes	1660	1232	721	23	52	288
1	Adelphi University	Yes	2186	1924	512	16	29	268
2	Adrian College	Yes	1428	1097	336	22	50	103

OLS Regression Results

```

=====
==
Dep. Variable:          Apps    R-squared:                0.7
55
Model:                  OLS    Adj. R-squared:           0.7
50
Method:                 Least Squares    F-statistic:              18
0.5
Date:                  Mon, 22 Dec 2025    Prob (F-statistic):       1.90e-2
22
Time:                  10:18:05    Log-Likelihood:           -697
5.0
No. Observations:      777    AIC:                      1.398e+
04
Df Residuals:          763    BIC:                      1.404e+
04
Df Model:              13
Covariance Type:       nonrobust
=====

```

```

=====
===
              coef      std err          t      P>|t|      [0.025      0.9
75]
-----
---
const      -3211.5706    692.943     -4.635     0.000    -4571.872    -1851.
270
Top10perc    21.3943     10.228      2.092     0.037      1.317      41.
472
Top25perc   -2.6605      8.270     -0.322     0.748     -18.895      13.
574
F.Undergrad  0.6588      0.020     32.148     0.000      0.619       0.
699
P.Undergrad -0.0841      0.059     -1.422     0.155     -0.200       0.
032
Outstate     0.0202      0.033      0.617     0.537     -0.044       0.
084
Room.Board   0.3437      0.088      3.913     0.000      0.171       0.
516
Personal    -0.1526      0.115     -1.325     0.186     -0.379       0.
073
PhD          0.8326      8.447      0.099     0.922     -15.749      17.
414
Terminal    -7.4478      9.331     -0.798     0.425     -25.765      10.
869
S.F.Ratio   33.5019     23.807      1.407     0.160     -13.234      80.
238
perc.alumni -26.8580      7.470     -3.595     0.000     -41.523     -12.
194
Expend       0.1072      0.023      4.706     0.000      0.062       0.
152
Grad.Rate   23.5055      5.420      4.337     0.000     12.866      34.
145
=====

```

```

==
Omnibus:              1050.898    Durbin-Watson:           1.8

```



```

27
Prob(Omnibus):                0.000    Jarque-Bera (JB):                430238.0
54
Skew:                        6.780    Prob(JB):                        0.
00
Kurtosis:                    117.479    Cond. No.                        1.64e+
05
=====
==

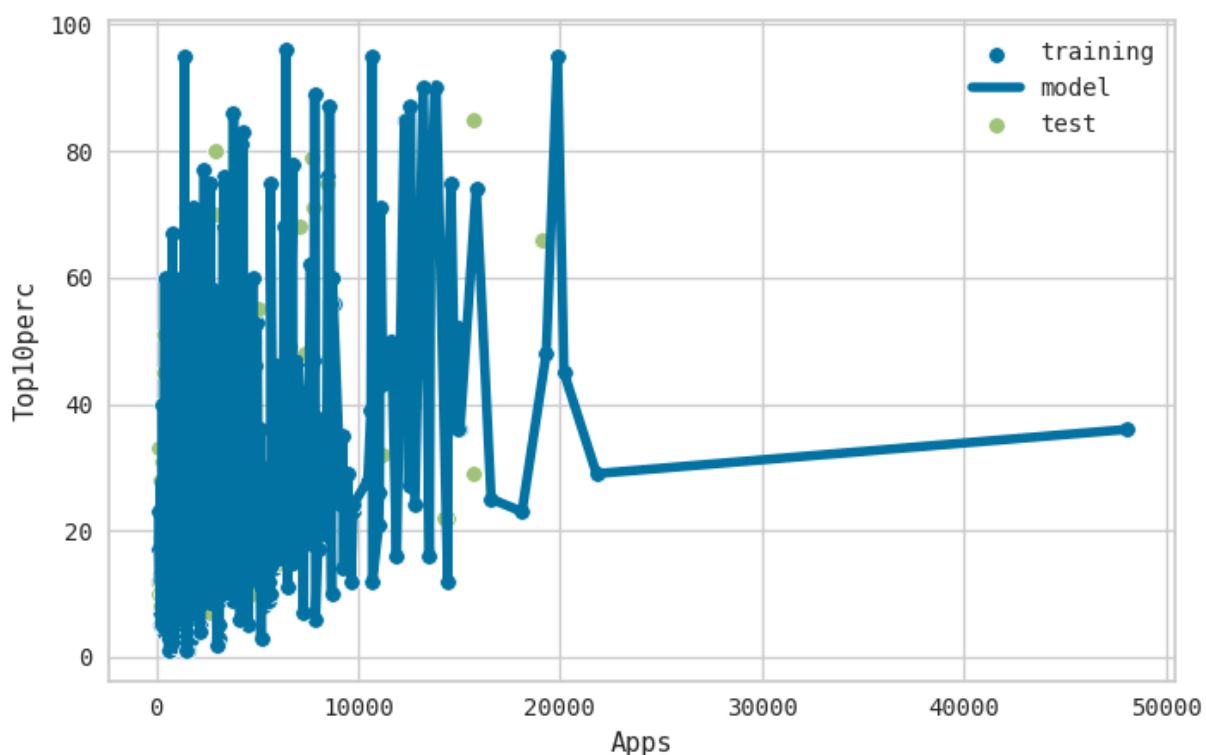
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 1.64e+05. This might indicate that there are strong multicollinearity or other numerical problems.

```
Out[55]: <function matplotlib.pyplot.show(close=None, block=None)>
```

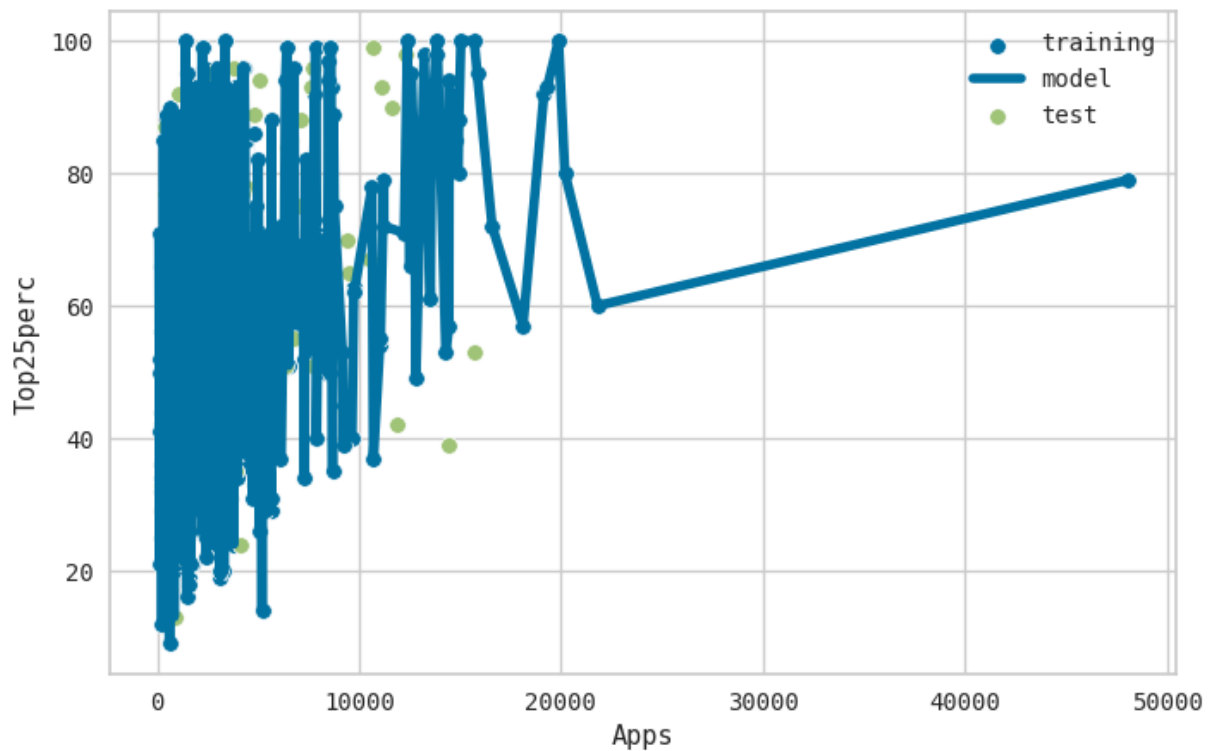


```

In [56]: col = 'Apps'
target = 'Top25perc'
x_train, x_test, y_train, y_test = train_test_split(College[col], College[target],
var = sm.add_constant(x_train, prepend=True)
model = sm.OLS(y_train, var)
res = model.fit()
pred = res.predict(sm.add_constant(x_test, prepend=True))
fig, axs = plt.subplots(figsize=(8,5))
sns.scatterplot(x=x_train, y=y_train, ax=axs, alpha=1, label='training')
sns.lineplot(x=x_train, y = y_train, ax = axs, label='model', linewidth = 4)
sns.scatterplot(x=x_test, y=y_test, ax=axs, label='test')
plt.show

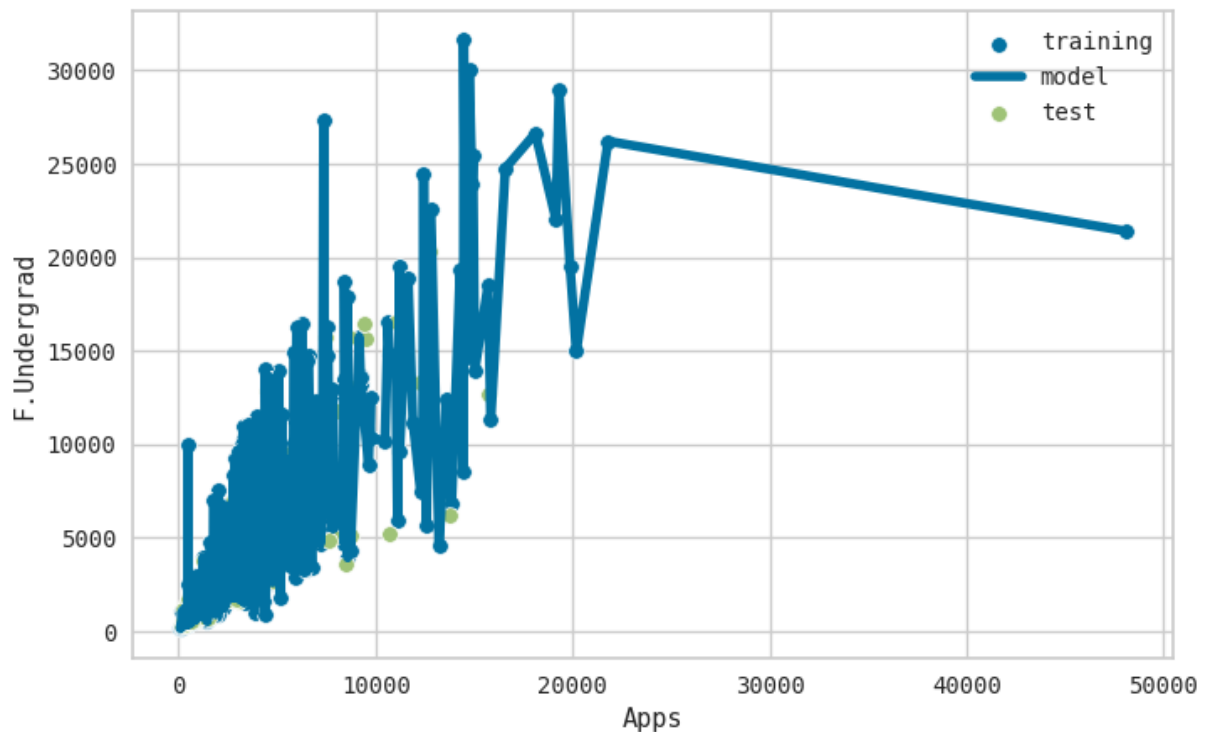
```

```
Out[56]: <function matplotlib.pyplot.show(close=None, block=None)>
```



```
In [57]: col = 'Apps'
target = 'F.Undergrad'
x_train, x_test, y_train, y_test = train_test_split(College[col], College[target])
var = sm.add_constant(x_train, prepend=True)
model = sm.OLS(y_train, var)
res = model.fit()
pred = res.predict(sm.add_constant(x_test, prepend=True))
fig, axs = plt.subplots(figsize=(8,5))
sns.scatterplot(x=x_train, y=y_train, ax=axs, alpha=1, label='training')
sns.lineplot(x=x_train, y = y_train, ax = axs, label='model', linewidth = 4)
sns.scatterplot(x=x_test, y=y_test, ax=axs, label='test')
plt.show
```

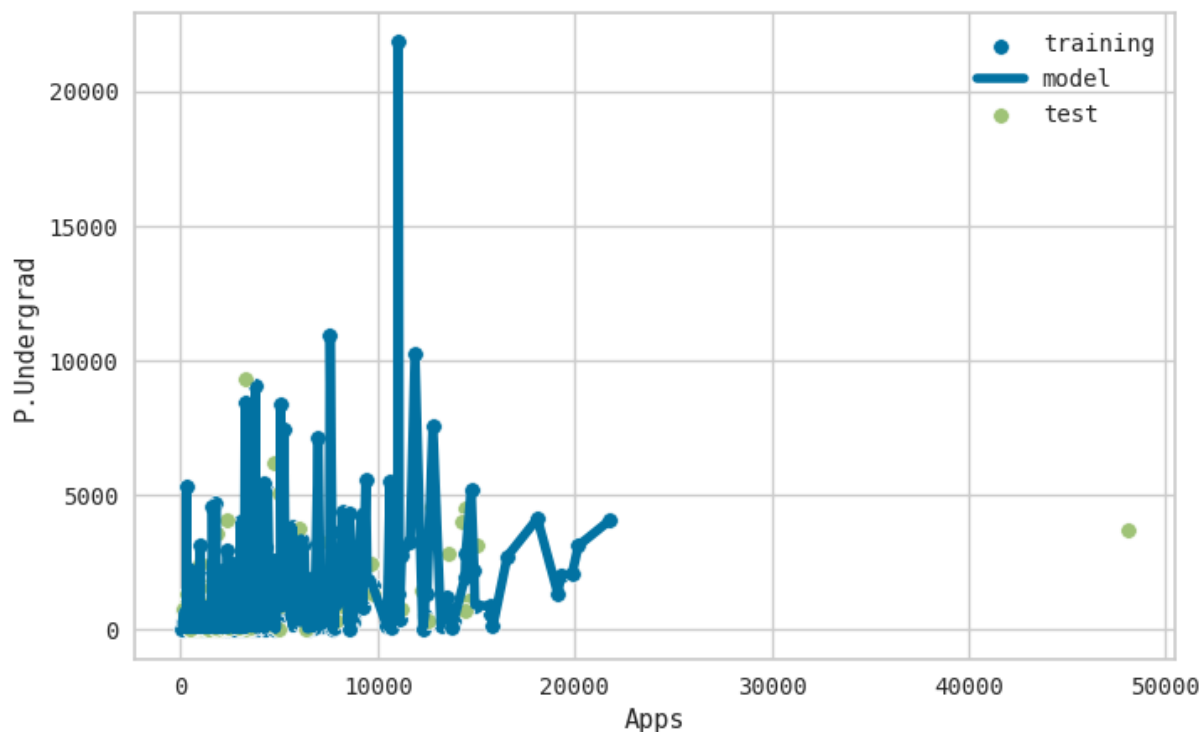
```
Out[57]: <function matplotlib.pyplot.show(close=None, block=None)>
```



```
In [58]: col = 'Apps'
target = 'P.Undergrad'
x_train, x_test, y_train, y_test = train_test_split(College[col], College[target],
var = sm.add_constant(x_train,prepend=True)
model = sm.OLS(y_train, var)
res = model.fit()
pred = res.predict(sm.add_constant(x_test, prepend=True))
fig, axs = plt.subplots(figsize=(8,5))
sn.scatterplot(x=x_train, y=y_train, ax=axs, alpha=1, label='training')
sn.lineplot(x=x_train, y = y_train, ax = axs, label='model', linewidth = 4)
sn.scatterplot(x=x_test, y=y_test, ax=axs, label='test')

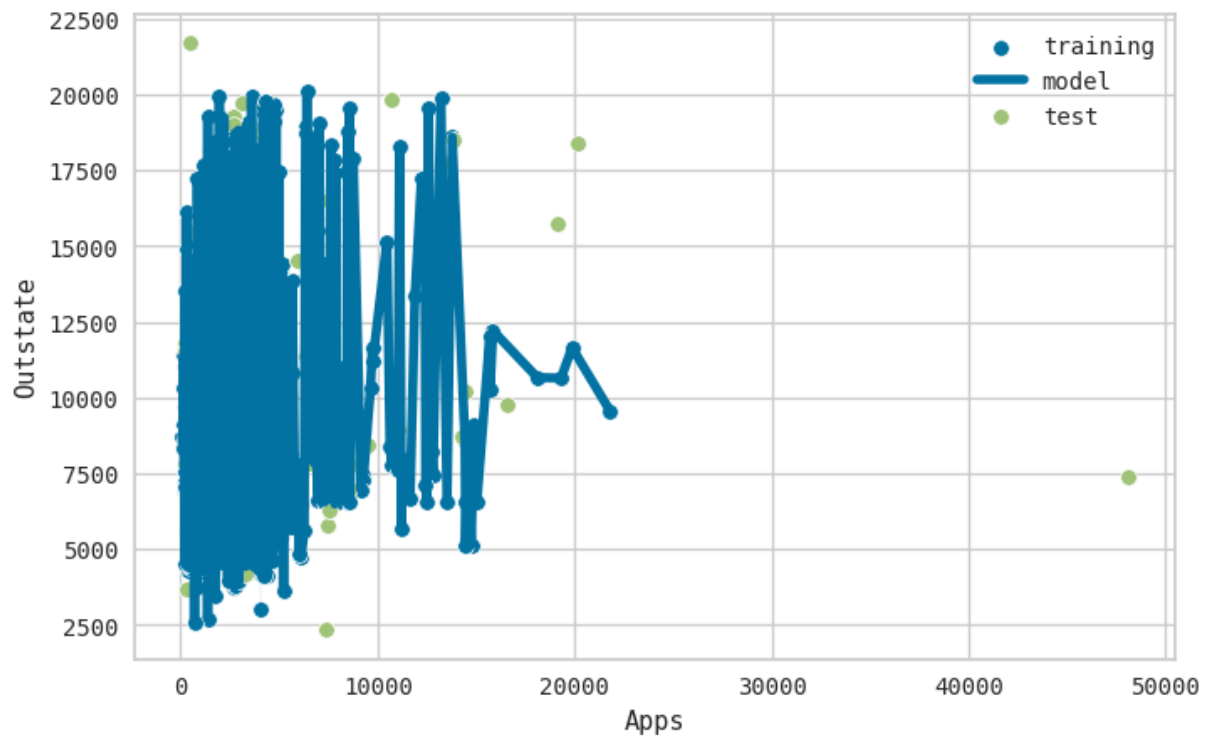
plt.show
```

```
Out[58]: <function matplotlib.pyplot.show(close=None, block=None)>
```



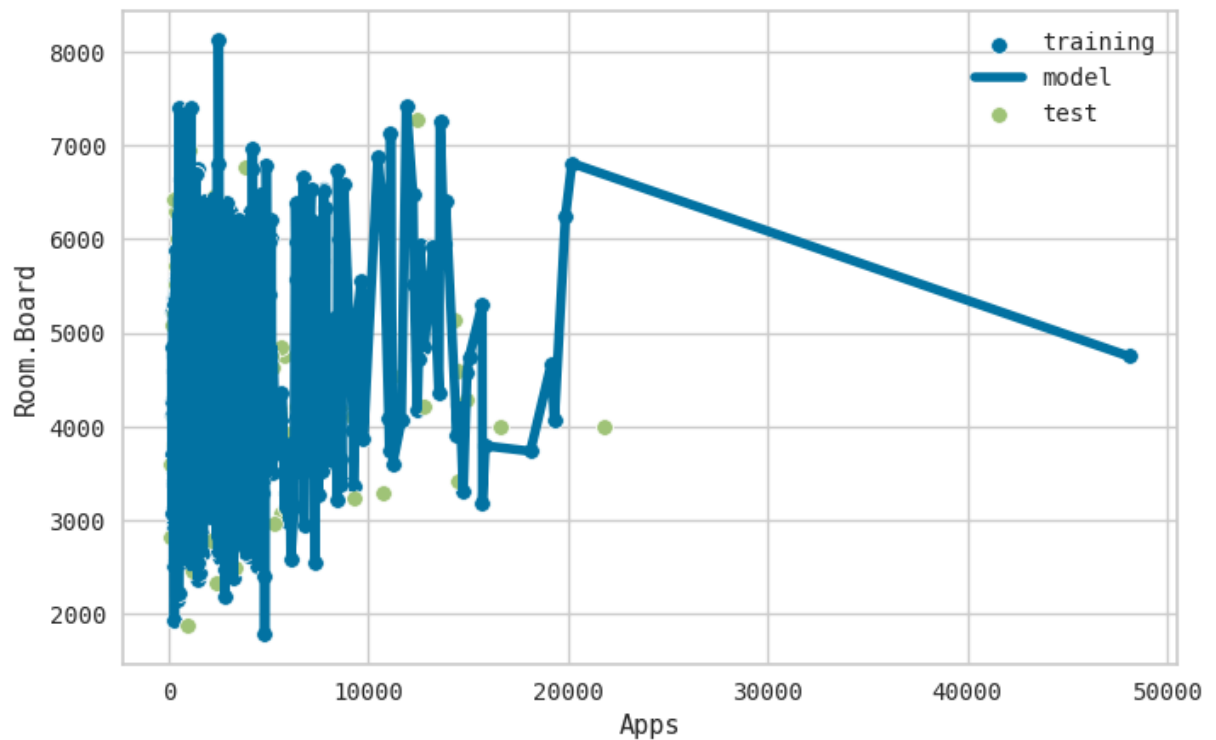
```
In [59]: col = 'Apps'
target = 'Outstate'
x_train, x_test, y_train, y_test = train_test_split(College[col], College[target],
var = sm.add_constant(x_train,prepend=True)
model = sm.OLS(y_train, var)
res = model.fit()
pred = res.predict(sm.add_constant(x_test, prepend=True))
fig, axs = plt.subplots(figsize=(8,5))
sn.scatterplot(x=x_train, y=y_train, ax=axs, alpha=1, label='training')
sn.lineplot(x=x_train, y = y_train, ax = axs, label='model', linewidth = 4)
sn.scatterplot(x=x_test, y=y_test, ax=axs, label='test')
```

```
Out[59]: <Axes: xlabel='Apps', ylabel='Outstate'>
```



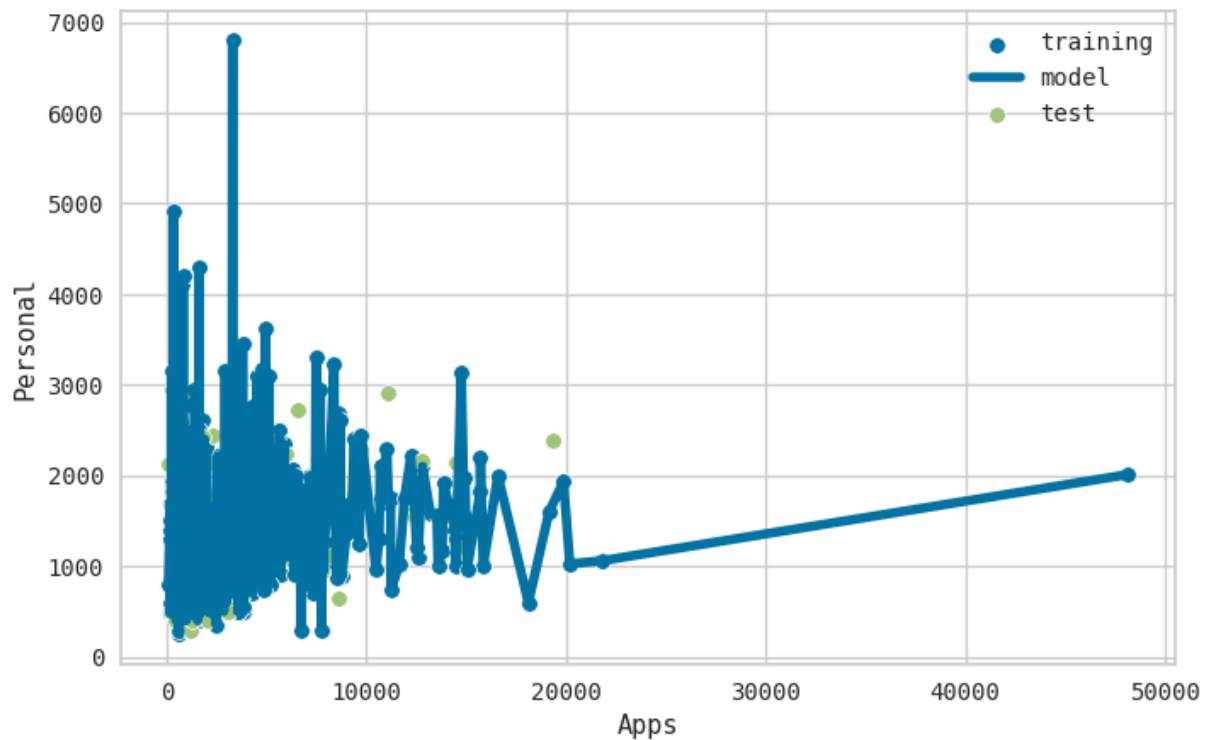
```
In [60]: col = 'Apps'
target = 'Room.Board'
x_train, x_test, y_train, y_test = train_test_split(College[col], College[target],
var = sm.add_constant(x_train,prepend=True)
model = sm.OLS(y_train, var)
res = model.fit()
pred = res.predict(sm.add_constant(x_test, prepend=True))
fig, axs = plt.subplots(figsize=(8,5))
sn.scatterplot(x=x_train, y=y_train, ax=axs, alpha=1, label='training')
sn.lineplot(x=x_train, y = y_train, ax = axs, label='model', linewidth = 4)
sn.scatterplot(x=x_test, y=y_test, ax=axs, label='test')
```

```
Out[60]: <Axes: xlabel='Apps', ylabel='Room.Board'>
```



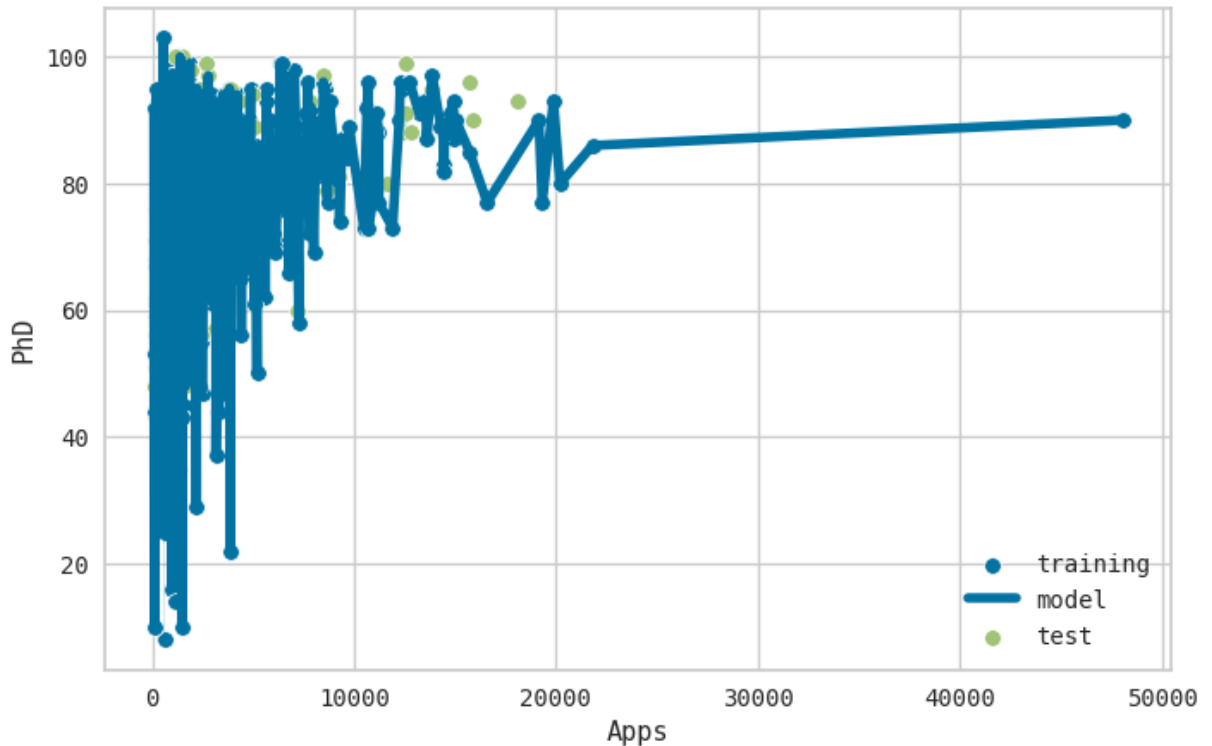
```
In [61]: col = 'Apps'
target = 'Personal'
x_train, x_test, y_train, y_test = train_test_split(College[col], College[target],
var = sm.add_constant(x_train,prepend=True)
model = sm.OLS(y_train, var)
res = model.fit()
pred = res.predict(sm.add_constant(x_test, prepend=True))
fig, axs = plt.subplots(figsize=(8,5))
sn.scatterplot(x=x_train, y=y_train, ax=axs, alpha=1, label='training')
sn.lineplot(x=x_train, y = y_train, ax = axs, label='model', linewidth = 4)
sn.scatterplot(x=x_test, y=y_test, ax=axs, label='test')
```

Out[61]: <Axes: xlabel='Apps', ylabel='Personal'>



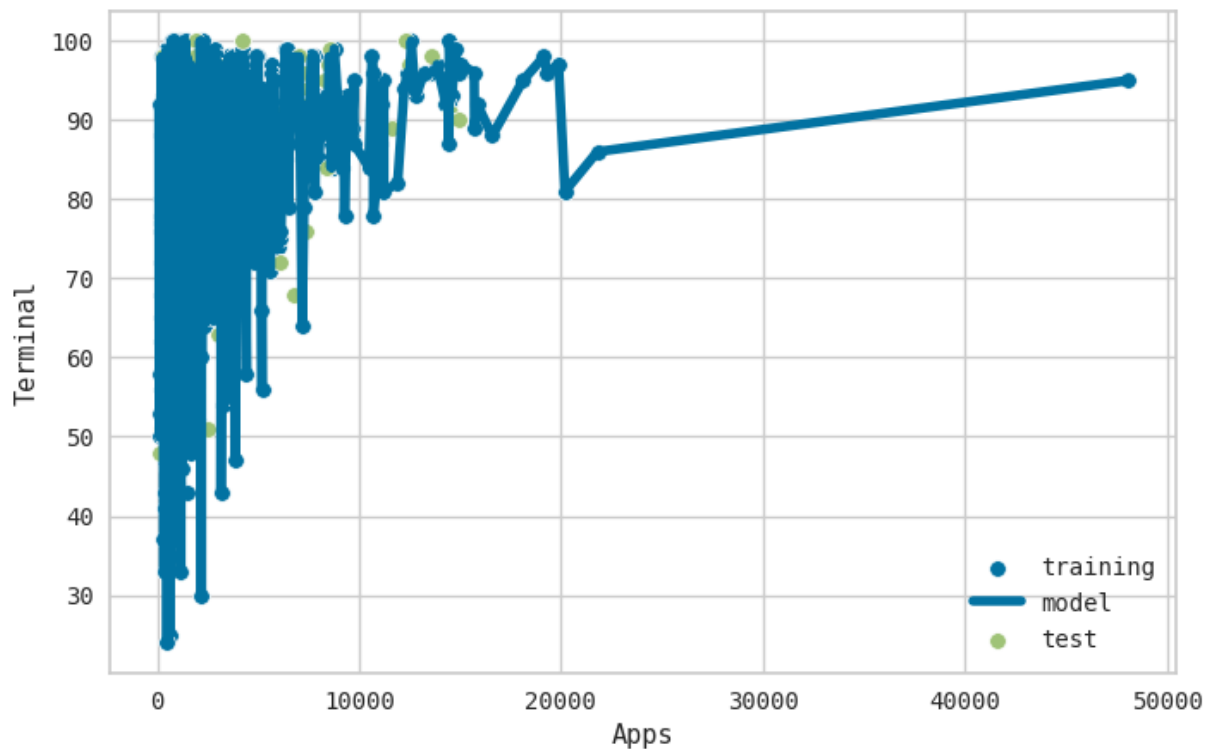
```
In [62]: col = 'Apps'
target = 'PhD'
x_train, x_test, y_train, y_test = train_test_split(College[col], College[target],
var = sm.add_constant(x_train,prepend=True)
model = sm.OLS(y_train, var)
res = model.fit()
pred = res.predict(sm.add_constant(x_test, prepend=True))
fig, axs = plt.subplots(figsize=(8,5))
sn.scatterplot(x=x_train, y=y_train, ax=axs, alpha=1, label='training')
sn.lineplot(x=x_train, y = y_train, ax = axs, label='model', linewidth = 4)
sn.scatterplot(x=x_test, y=y_test, ax=axs, label='test')
```

Out[62]: <Axes: xlabel='Apps', ylabel='PhD'>



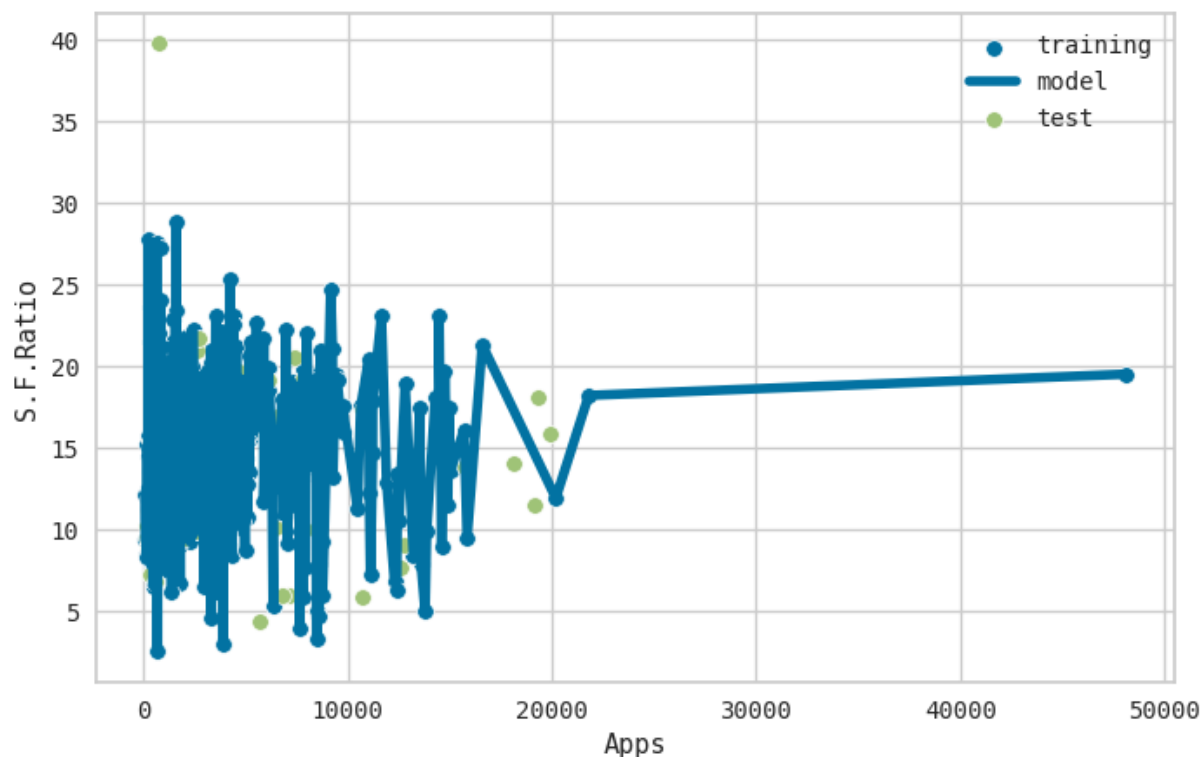
```
In [63]: col = 'Apps'
target = 'Terminal'
x_train, x_test, y_train, y_test = train_test_split(College[col], College[target],
var = sm.add_constant(x_train,prepend=True))
model = sm.OLS(y_train, var)
res = model.fit()
pred = res.predict(sm.add_constant(x_test, prepend=True))
fig, axs = plt.subplots(figsize=(8,5))
sns.scatterplot(x=x_train, y=y_train, ax=axs, alpha=1, label='training')
sns.lineplot(x=x_train, y = y_train, ax = axs, label='model', linewidth = 4)
sns.scatterplot(x=x_test, y=y_test, ax=axs, label='test')
```

```
Out[63]: <Axes: xlabel='Apps', ylabel='Terminal'>
```

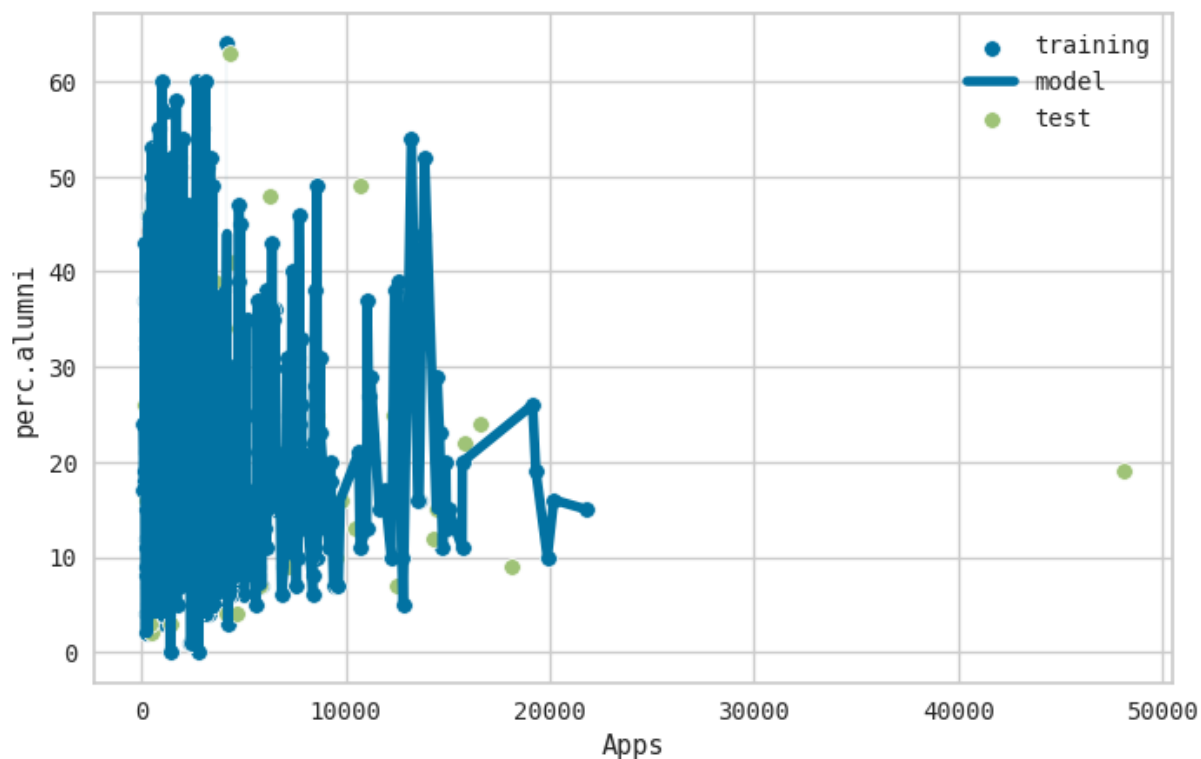
```
In [64]: col = 'Apps'
target = 'S.F.Ratio'
x_train, x_test, y_train, y_test = train_test_split(College[col], College[target],
var = sm.add_constant(x_train, prepend=True))
model = sm.OLS(y_train, var)
res = model.fit()
pred = res.predict(sm.add_constant(x_test, prepend=True))
fig, axs = plt.subplots(figsize=(8,5))
sns.scatterplot(x=x_train, y=y_train, ax=axs, alpha=1, label='training')
sns.lineplot(x=x_train, y = y_train, ax = axs, label='model', linewidth = 4)
sns.scatterplot(x=x_test, y=y_test, ax=axs, label='test')
```

```
Out[64]: <Axes: xlabel='Apps', ylabel='S.F.Ratio'>
```



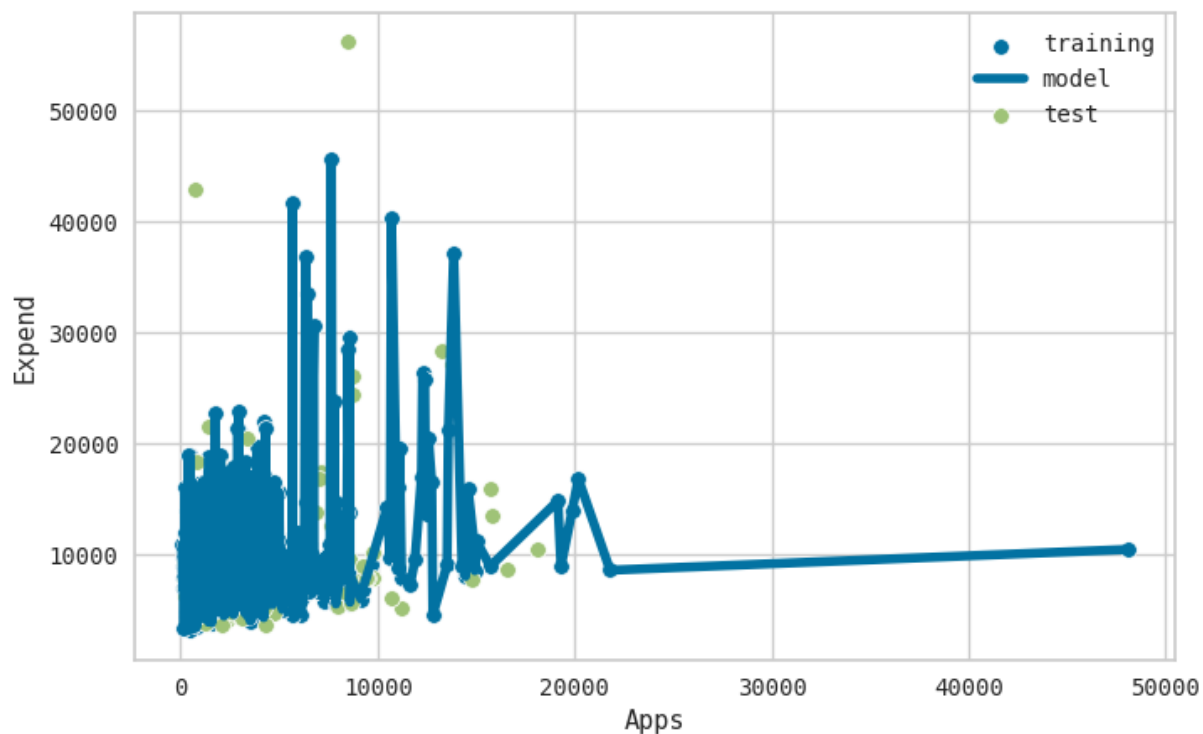
```
In [65]: col = 'Apps'
target = 'perc.alumni'
x_train, x_test, y_train, y_test = train_test_split(College[col], College[target],
var = sm.add_constant(x_train,prepend=True)
model = sm.OLS(y_train, var)
res = model.fit()
pred = res.predict(sm.add_constant(x_test, prepend=True))
fig, axs = plt.subplots(figsize=(8,5))
sn.scatterplot(x=x_train, y=y_train, ax=axs, alpha=1, label='training')
sn.lineplot(x=x_train, y = y_train, ax = axs, label='model', linewidth = 4)
sn.scatterplot(x=x_test, y=y_test, ax=axs, label='test')
```

```
Out[65]: <Axes: xlabel='Apps', ylabel='perc.alumni'>
```



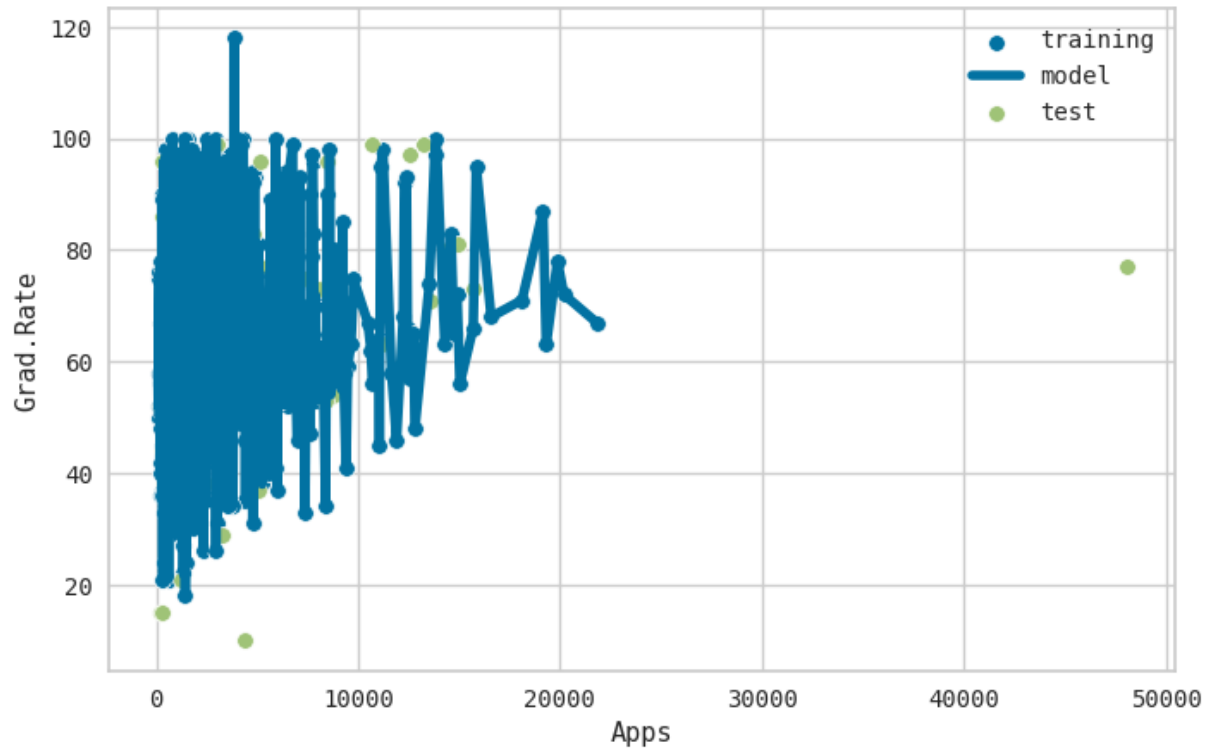
```
In [68]: col = 'Apps'
target = 'Expend'
x_train, x_test, y_train, y_test = train_test_split(College[col], College[target],
                                                    test_size=0.2, random_state=42)
var = sm.add_constant(x_train, prepend=True)
model = sm.OLS(y_train, var)
res = model.fit()
pred = res.predict(sm.add_constant(x_test, prepend=True))
fig, axs = plt.subplots(figsize=(8,5))
sn.scatterplot(x=x_train, y=y_train, ax=axs, alpha=1, label='training')
sn.lineplot(x=x_train, y = y_train, ax = axs, label='model', linewidth = 4)
sn.scatterplot(x=x_test, y=y_test, ax=axs, label='test')
```

```
Out[68]: <Axes: xlabel='Apps', ylabel='Expend'>
```



```
In [67]: col = 'Apps'
target = 'Grad.Rate'
x_train, x_test, y_train, y_test = train_test_split(College[col], College[target],
var = sm.add_constant(x_train,prepend=True)
model = sm.OLS(y_train, var)
res = model.fit()
pred = res.predict(sm.add_constant(x_test, prepend=True))
fig, axs = plt.subplots(figsize=(8,5))
sn.scatterplot(x=x_train, y=y_train, ax=axs, alpha=1, label='training')
sn.lineplot(x=x_train, y = y_train, ax = axs, label='model', linewidth = 4)
sn.scatterplot(x=x_test, y=y_test, ax=axs, label='test')
```

```
Out[67]: <Axes: xlabel='Apps', ylabel='Grad.Rate'>
```



a) die Werte mit einem niedrigen P-Wert welche nahe null ist sind dabei signifikant Dabei sind die drei wichtigsten P.Undergrad, S.F.Ratio, Top25perc

c) das R Squared Ajdusted bedeutet, wie viel Prozent sich von Target durch Col erklären lassen bei mehreren Variablen

In []:

In []: